

Git Life Cycle Process & Git Ignore

Git is used in our day-to-day work, we use git for keeping a track of our files, working in a collaboration with our team, to go back to our previous code versions if we face some error.

Remember that each file in your working directory can be in one of two states: **tracked or untracked**.

1. Tracked files

Tracked files are files that Git knows about it. Files that are previous git snapshot are called tracked files. Tracked files can be in one of following states.

- a. Committed
- b. Modified
- c. Staging

Committed state

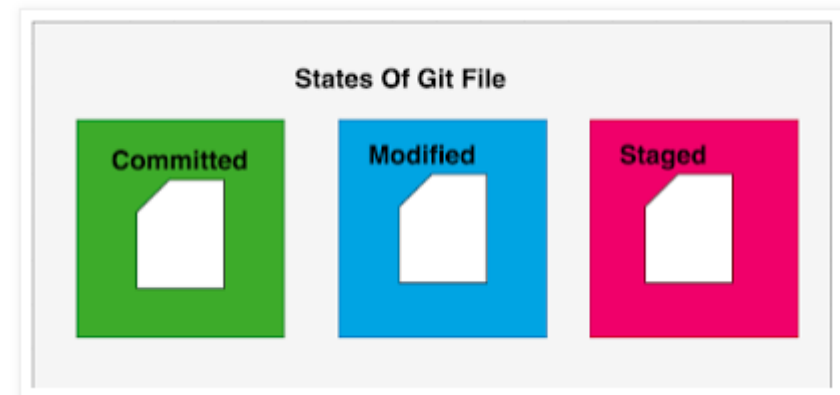
Data is successfully stored into local database.

Modified state

File is changed, but the file data is not yet saved to local database.

Staging state

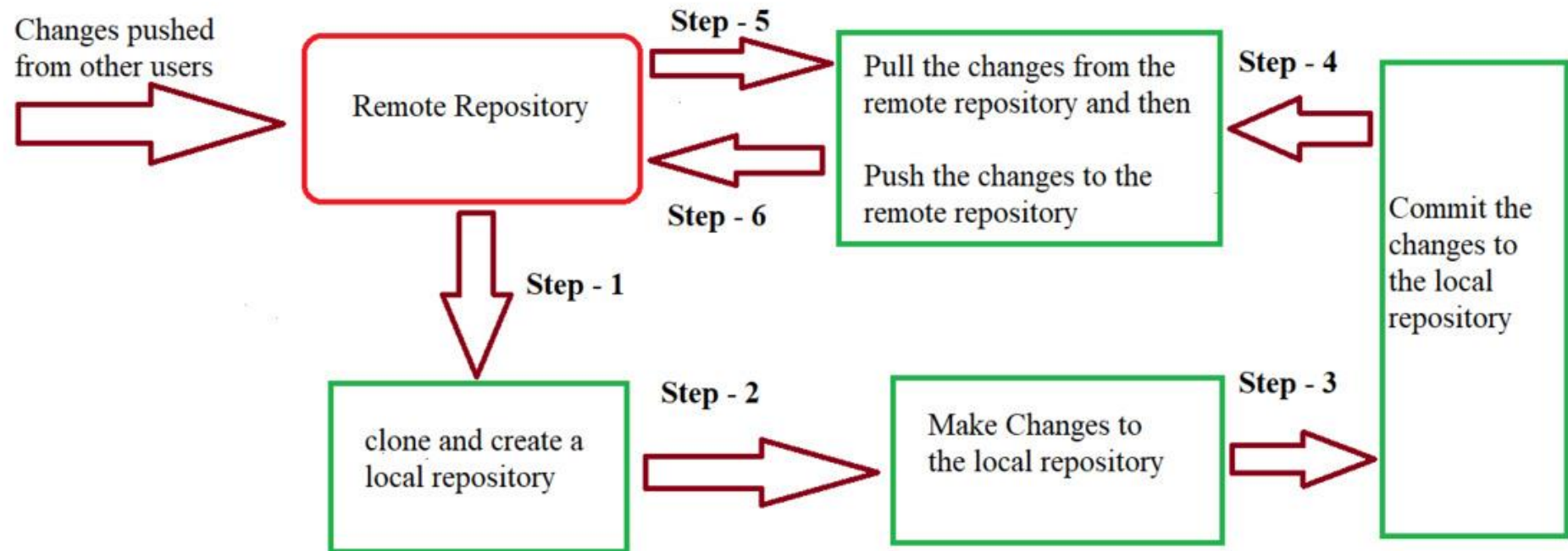
When you mark a file (or) files to go to your next commit state, those files are in staging state.



2. Untracked files

When a new file is added to the git repository it comes under untracked category. Whenever git sees a new file that is not in previous snapshot, then git treats this as untracked file. so in short , **Untracked files are everything else — any files in your working directory that were not in your last snapshot and are not in your staging area.**

Let us look at the Life Cycle that git has and understand more about its life cycle.



- **In Step – 1**, We first clone any of the code residing in the remote repository to make our own local repository.
- **In Step-2** we edit the files that we have cloned in our local repository and make the necessary changes in it.
- **In Step-3** we commit our changes by first adding them to our staging area and committing them with a commit message.
- **In Step – 4 and Step-5** we first check whether there are any of the changes done in the remote repository by some other users and we first pull that changes.
- If there are no changes we directly proceed with **Step – 6** in which we push our changes to the remote repository and we are done with our work.

What is gitignore?

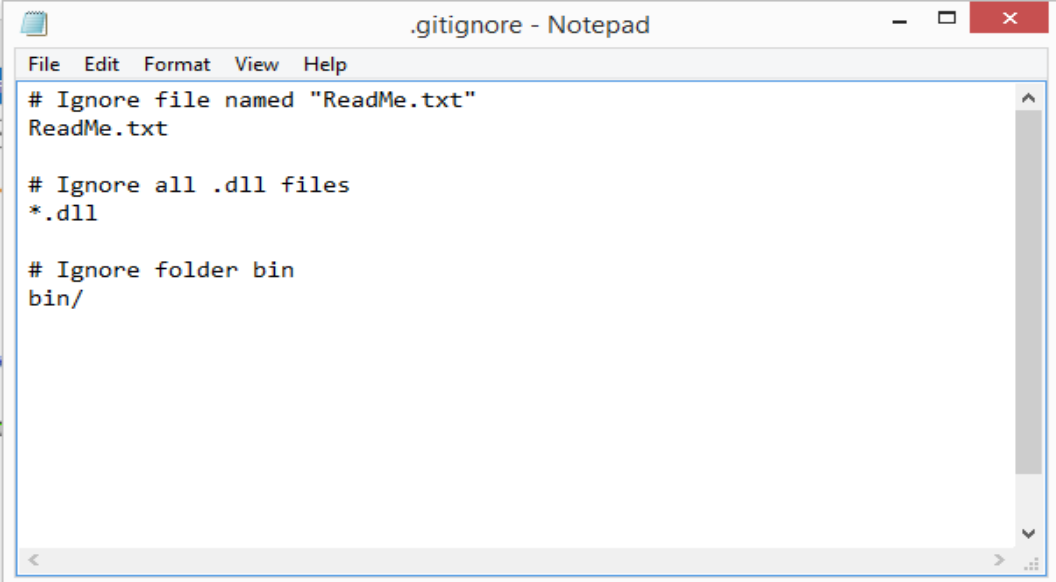
When you're working in git , it watches every file in and considers it in three ways:

Tracked: You've already staged or committed the file.

Untracked: You've not staged or committed.

Ignored: You've explicitly told Git to ignore the file(s).

The .gitignore file tells Git which files to ignore when committing your project to the GitHub repository. gitignore is located in the root directory of your repo. The .gitignore file itself is a plain text document.



```
File Edit Format View Help
# Ignore file named "ReadMe.txt"
ReadMe.txt

# Ignore all .* files
*.*

# Ignore folder bin
bin/
```

Git ignore patterns

.gitignore uses globbing patterns to match against file names. You can construct your patterns using various symbols:

Note: we use a single .gitignore which applies to the entire repository. It is also possible to have additional .gitignore files in subdirectories. These only apply to files or folders within that directory.

Pattern	Explanation/Matches	Examples
	Blank lines are ignored	
<i># text comment</i>	Lines starting with # are ignored	
<i>name</i>	All <i>name</i> files, <i>name</i> folders, and files and folders in any <i>name</i> folder	/name.log /name/file.txt /lib/name.log
<i>name/</i>	Ending with / specifies the pattern is for a folder. Matches all files and folders in any <i>name</i> folder	/name/file.txt /name/log/name.log no match: /name.log
<i>name.file</i>	All files with the <i>name.file</i>	/name.file /lib/name.file

<i>/name.file</i>	Starting with / specifies the pattern matches only files in the root folder	/name.file no match: /lib/name.file
<i>lib/name.file</i>	Patterns specifying files in specific folders are always relative to root (even if you do not start with /)	/lib/name.file no match: name.file /test/lib/name.file
<i>**/lib/name.file</i>	Starting with ** before / specifies that it matches any folder in the repository. Not just on root.	/lib/name.file /test/lib/name.file
<i>**/name</i>	All <i>name</i> folders, and files and folders in any <i>name</i> folder	/name/log.file /lib/name/log.file /name/lib/log.file
<i>/lib/**/*.name</i>	All <i>name</i> folders, and files and folders in any <i>name</i> folder within the lib folder.	/lib/name/log.file /lib/test/name/log.file /lib/test/ver1/name/log.file no match: /name/log.file

<i>*.file</i>	All files with the <i>.file</i> extension	/name.file /lib/name.file
<i>*name/</i>	All folders ending with <i>name</i>	/lastname/log.file /firstname/log.file
<i>name?.file</i>	? matches a single non-specific character	/names.file /name1.file no match: /names1.file
<i>name[a-z].file</i>	[<i>range</i>] matches a single character in the specified range (in this case a character in the range of a-z, and also be numeric.)	/names.file /nameb.file no match: /name1.file
<i>name[abc].file</i>	[<i>set</i>] matches a single character in the specified set of characters (in this case either a, b, or c)	/namea.file /nameb.file no match: /names.file

<i>name</i> [!abc]. <i>file</i>	[!set] matches a single character, except the ones specified in the set of characters (in this case a, b, or c)	/names. <i>file</i> /namex. <i>file</i> no match: /namesb. <i>file</i>
*. <i>file</i>	All files with the <i>.file</i> extension	/name. <i>file</i> /lib/name. <i>file</i>
<i>name</i> / ! <i>name</i> /secret.log	! specifies a negation or exception. Matches all files and folders in any <i>name</i> folder, except name/secret.log	/name/ <i>file</i> .txt /name/log/name.log no match: /name/secret.log
*. <i>file</i> ! <i>name</i> . <i>file</i>	! specifies a negation or exception. All files with the <i>.file</i> extension, except name. <i>file</i>	/log. <i>file</i> /lastname. <i>file</i> no match: /name. <i>file</i>

For More information Visit: <https://github.com/github/gitignore>